Python Concepts — A Practical, Plain-Language Guide

Indentation & Comments · Functions · OOP · File Handling · Exception Handling · NumPy & Pandas · Selenium Scraping · Regex · Multithreading/Multiprocessing · Concurrency vs Parallelism

Prepared for Gitesh — a simple-language reference with examples, diagrams, and quick self-check questions.

Contents

1. Indentation & Comments

What is Indentation?

Python uses **spacing** instead of curly braces `{ }` to show which code belongs together. If the spacing is wrong, Python will not run the code — it's not just a style choice, it's a rule.

Think of it like... An outline in a notebook. Main points start at the margin, sub-points are indented under them. Mixing that up makes the outline confusing — Python feels the same way.

```
def greet(name):
    if name:
        print(f"Hello, {name}!")   # this line is indented 8 spaces (inside if, inside def)
    else:
        print("Hello, stranger!")
```

Common mistake Never mix tabs and spaces in the same file. Pick spaces (4 is standard) and stay consistent.

What are Comments?

Comments are notes for humans — Python ignores them completely.

```
# This is a single-line comment
print("Hi")  # comment after code is also fine

"""
This is a multi-line comment (technically a string),
often used to describe what a function does.
"""
```

Good habit Write comments that explain *why* you did something, not *what* the code does — the code already shows "what".

Quick check: What happens if you indent one line of an `if` block with 4 spaces and the next with 5?
Answer: Python raises an `IndentationError` — every line in the same block must match exactly.

2. Functions

A function is a reusable block of code that performs a task. Instead of writing the same steps again and again, you write them once and "call" them whenever needed.

```
def add_numbers(a, b):
    """Returns the sum of a and b."""
    return a + b

result = add_numbers(5, 3)
print(result)  # 8
```

Key Building Blocks

Concept	Meaning	Example
Parameters	Inputs a function accepts	`def add_numbers(a, b):`
Return value	Output sent back	`return a + b`
Default argument	Value used if none is given	`def greet(name="Guest"):`
*args / **kwargs	Accept any number of extra arguments	`def f(*args, **kwargs):`
Lambda	A tiny, unnamed function	`square = lambda x: x*x`

Think of it like...A coffee machine. You put in inputs (water, beans = parameters), it processes them, and gives you an output (coffee = return value). You don't rebuild the machine every time you want coffee.

Quick check: What's the difference between `print()` inside a function and `return`?
Answer: `print()` just displays a value; `return` actually sends the value back so it can be stored or reused.

3. Object-Oriented Programming (OOP)

OOP is a way of organizing code around **objects** — bundles of data (attributes) and behavior (methods) modeled on real-world things.

```
class Car:
    def __init__(self, brand, speed=0):
        self.brand = brand      # attribute
        self.speed = speed

    def accelerate(self, amount):   # method
        self.speed += amount
        print(f"{self.brand} is now going {self.speed} km/h")

my_car = Car("Toyota")
my_car.accelerate(20)   # Toyota is now going 20 km/h
```

The Four Pillars

Pillar	Plain-language meaning
Encapsulation	Keep data and the code that uses it bundled together, hiding internal details
Inheritance	A new class can reuse and extend an existing class
Polymorphism	Different classes can respond to the same method call in their own way
Abstraction	Show only what's necessary; hide the complex internal logic

```
class ElectricCar(Car):          # Inheritance: reuses Car
    def accelerate(self, amount): # Polymorphism: own version of the method
        self.speed += amount * 2  # electric cars accelerate faster!
        print(f"{self.brand} (EV) zips to {self.speed} km/h")
```

Think of it like...A "Car" blueprint (class) used to build actual cars (objects). Each car built from the blueprint has its own brand and speed, but they all share the same design.

4. File Handling

File handling lets your program read data from files or save data into them — so information isn't lost when the program ends.

```
# Writing to a file
with open("notes.txt", "w") as f:
    f.write("Hello, this is saved on disk.\n")

# Reading from a file
with open("notes.txt", "r") as f:
    content = f.read()
    print(content)
```

Common Modes

Mode	Meaning
`"r"`	Read (file must exist)
`"w"`	Write (creates new file / erases old content)
`"a"`	Append (adds to the end without erasing)
`"rb" / "wb"`	Read/write in binary mode (images, PDFs, etc.)

Why use "with"? The `with` keyword automatically closes the file for you, even if an error happens in between. Without it, you'd have to remember to call `f.close()` yourself.

5. Exception Handling

Errors happen — bad input, a missing file, dividing by zero. Exception handling lets your program respond gracefully instead of crashing.

```
try:
    number = int(input("Enter a number: "))
    result = 10 / number
except ValueError:
    print("That's not a valid number.")
except ZeroDivisionError:
    print("You can't divide by zero.")
else:
    print(f"Result is {result}")   # runs only if no error occurred
finally:
    print("Done trying.")           # always runs, error or not
```

Think of it like...A safety net under a tightrope walker. The walker (your code) tries to cross; if they slip (an error), the net (`except`) catches them instead of letting them crash to the floor.

Best practice Catch specific exceptions (like `ValueError`) rather than a bare `except:` — that way you don't accidentally hide bugs you didn't expect.

6. NumPy & Pandas

NumPy — Fast Number Crunching

NumPy provides the **array**, a fast way to store and calculate on large sets of numbers — much quicker than plain Python lists.

```
import numpy as np

arr = np.array([1, 2, 3, 4])
print(arr * 2)          # [2 4 6 8]  -- applies to every element at once
print(arr.mean())       # 2.5
```

Pandas — Working with Tables

Pandas gives you the **DataFrame** — think of it as a spreadsheet inside Python, with rows, columns, and labels.

```
import pandas as pd

data = {"Name": ["Amit", "Priya"], "Age": [28, 24]}
df = pd.DataFrame(data)

print(df[df["Age"] > 25])   # filter rows where Age > 25
df.to_csv("people.csv", index=False)   # save to a file
```

NumPy	Pandas
Best for numeric arrays & math	Best for labeled, table-like data
Core structure: `ndarray`	Core structure: `DataFrame` / `Series`
Used for calculations, image data, matrices	Used for CSVs, Excel files, data cleaning, analysis

Good to know Pandas is actually built on top of NumPy — a DataFrame's columns are stored internally as NumPy arrays.

7. Selenium for Web Scraping

Selenium automates a real web browser — clicking buttons, filling forms, and reading page content — which is useful for sites that load data dynamically with JavaScript (where simple request-based scraping falls short).

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get("https://example.com")

heading = driver.find_element(By.TAG_NAME, "h1")
print(heading.text)

driver.quit()
```

Typical Workflow

Be responsible Always check a site's `robots.txt` and terms of service before scraping, and avoid hammering a server with rapid, repeated requests.

8. Regular Expressions (Regex)

Regex is a compact language for finding patterns inside text — like "find anything that looks like an email address" instead of checking character-by-character yourself.

```
import re

text = "Contact us at support@example.com or sales@example.com"
emails = re.findall(r"[\w.+-]+@[\w-]+\.[a-zA-Z]{2,}", text)
print(emails)   # ['support@example.com', 'sales@example.com']

# Replace digits with '#'
masked = re.sub(r"\d", "#", "Call 9876543210")
print(masked)   # Call ##########
```

Handy Symbols Cheat-Sheet

Symbol	Meaning
`\d`	Any digit (0-9)
`\w`	Any letter, digit, or underscore
`\s`	Any whitespace (space, tab)
`+`	One or more of the previous item
`*`	Zero or more of the previous item
`{2,}`	Two or more repetitions
`^ / $`	Start / end of the string

> **Think of it like...**A search template with wildcards — like telling someone "find any 10-digit number" instead of listing every possible phone number yourself.

9. Multithreading & Multiprocessing

Both let a program do multiple things "at once" — but they achieve it very differently.

Multithreading

Runs multiple threads inside the **same process**, sharing the same memory. Great for tasks that spend time waiting (network calls, file I/O) because Python's Global Interpreter Lock (GIL) still limits true parallel CPU work.

```
import threading

def download(name):
    print(f"Downloading {name}...")

t1 = threading.Thread(target=download, args=("file1",))
t2 = threading.Thread(target=download, args=("file2",))
t1.start(); t2.start()
t1.join(); t2.join()
```

Multiprocessing

Runs multiple **separate processes**, each with its own memory and its own Python interpreter — bypassing the GIL. Great for CPU-heavy tasks (number crunching, image processing).

```
from multiprocessing import Process

def compute(n):
    print(sum(i*i for i in range(n)))

p1 = Process(target=compute, args=(1000000,))
p2 = Process(target=compute, args=(1000000,))
p1.start(); p2.start()
p1.join(); p2.join()
```

	Multithreading	Multiprocessing
Memory	Shared	Separate
Best for	I/O-bound tasks (waiting)	CPU-bound tasks (heavy computation)
Limited by GIL?	Yes	No
Overhead	Lower	Higher (new process = more memory)

Quick check: You're scraping 100 web pages that each take time to load — threads or processes?
Answer: Threads — this is I/O-bound (mostly waiting on the network), which is exactly what multithreading handles well.

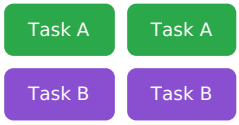
10. Concurrency vs Parallelism

These sound similar but describe different ideas.

Concurrency — juggling multiple tasks by switching between them

(one worker, switching back and forth)

Parallelism — actually running multiple tasks at the exact same time



(two workers, each running independently)

Think of it like...A single chef cooking two dishes by switching between pots (concurrency) versus two chefs each cooking one dish at the same time (parallelism).

	Concurrency	Parallelism
Definition	Dealing with multiple tasks by interleaving progress	Executing multiple tasks at the literal same instant
Needs multiple CPU cores?	No — works on a single core	Yes — requires multiple cores/CPUs
Python tool	`threading`, `asyncio`	`multiprocessing`
Goal	Responsiveness / not blocking	Raw speed for heavy computation

Key takeaway Concurrency is about *structure* (managing many tasks); parallelism is about *execution* (doing many things simultaneously). You can have concurrency without parallelism, but true parallelism always involves some concurrency in how tasks are organized.

End of guide — 10/10 topics covered.